WorkSkillsTrack Learner Support Portal

JavaScript Programmer – Two Month Integrated Project

Project Details

Learner name: Brian Mncube

Team names: Chad Teixeira & Modise Matjila

Assessor:

Project Start Date: 02 August 2026

Final Due Date: 02 September 2026

Month 1 Formative Questions (92 Marks)

1. Explain why the programming life cycle should be followed before coding begins. ***(2 marks)***

Following the life cycle makes sure you have a full understanding of how to get to the end-goal without too many challenges because most of the possible problems will be discovered and fixed from following the life cycle. This basically means fewer mistakes, and makes the finished product easier to test, fix, maintain, and understand.

2. List and explain the main steps of the programming life cycle. Your answer must show how each step contributes to the development of a working solution. ***(15 marks)***

The programming life cycle ensures software is built systematically and successfully. It begins with Problem Analysis, where developers gather requirements from stakeholders. This step ensures the final product solves the correct problem, preventing wasted effort. Next is Design, which creates the architectural blueprint, algorithms, and user interfaces. This roadmap prevents chaotic coding and ensures all components integrate smoothly.

The third step is Implementation, where the actual source code is written. This transforms the abstract design into a tangible, executable program. Fourth, Testing systematically identifies and removes bugs through unit, integration, and user acceptance testing. This guarantees the software is reliable, secure, and handles edge cases, turning a fragile script into a robust tool.

Finally, Deployment releases the finished product to users, while ongoing Maintenance monitors performance, fixes emerging issues, and adapts to changing needs. Maintenance ensures the solution remains functional and relevant over time.

3. Explain when const should be used instead of let. Also explain why var should normally be avoided in modern JavaScript. ***(5 marks)***

You should use const when a variable's value will not need to change after it is first set. Example: a fixed setting, or a value that is calculated once and never changed again. Use let when the value is expected to change while the program run. Example: a counter in a loop, or a total that keeps getting updated.
Var should be avoided in modern JavaScript for a few reasons: it's function-scoped (ignores blocks), allows redeclaration, and leaks into the global object—leading to unpredictable bugs. Use const by default for variables that don't change, and let only when reassignment is needed; both are block-scoped and safer.

4. Explain how local and global scope can affect the reliability and maintainability of a JavaScript application. ***(2 marks)***

Local variables are confined to their function or block, preventing accidental access or modification from elsewhere—this boosts reliability and reduces bugs. *Global variables*, by contrast, are accessible everywhere, inviting naming conflicts and unpredictable changes that make code harder to debug and maintain. *Best practice*: Keep variables local whenever possible and use global only when absolutely necessary.

5. Explain how map(), filter() and reduce() process an array of task objects differently. Provide one suitable use for each method. ***(6 marks)***

- map(): Changes every item in the array and returns a new array of the same length. Example: get just the titles of every task: tasks.map(task => task.title).
- filter(): Tests every item and keeps only the ones that pass, returning a new array that may be shorter than the original. Example: get only the completed tasks: tasks.filter(task => task.completed).
- reduce(): Combines every item down into one single value. Example: add up the total hours: tasks.reduce((sum, task) => sum + task.hours, 0)

6. Explain why an application should use classes or structured objects instead of storing related information in several unrelated variables. ***(5 marks)***

Objects and classes organize related data and behavior in one place, making code more readable and maintainable than scattered variables. A *class* acts as a reusable template, allowing you to create multiple objects without duplicating code—this prevents mismatched data, simplifies scaling, and mirrors real-world thinking. *Best practice*: Use classes/objects to bundle data together, making your code cleaner, safer, and easier for others to understand.

7. Explain how branches, pull requests and automated checks reduce risk when developers collaborate on a project. ***(5 marks)***

Branches: Let a developer work on a new feature or fix in an isolated copy of the code, without affecting the main code or other people's work, until it's ready to share.
Pull requests: A formal way to propose changes and have them reviewed by teammates before they're merged into the main branch. This catches mistakes early and keeps a record of what changed and why.
Automated checks: Tools like automated tests and linters, run through continuous Integration that automatically check new code before it can be merged, catching bugs early.

8. Study the following values: ***(8 marks total)***

```
const userName = “Brian";
const age = 24;
const isActive = true;
const selectedProject = null;
```

a. State the data type of each value. ***(4 marks)***

userName = “Brian" → String
age = 24 → Number
isActive = true → Boolean
selectedProject = null → Object

b. Explain how age could be converted from a number to a string. ***(1 mark)***

typeof checks and returns the data type of a value, as a string such as "string", "number", "boolean", "object", "undefined" or "function".

9. Analyse the following code: ***(8 marks total)***

```
const total = "10" + 5;
const looseComparison = 5 == "5";
const strictComparison = 5 === "5";
console.log(total);
console.log(looseComparison);
console.log(strictComparison);
```

a. State the output of each console.log() statement. ***(3 marks)***

1. “105”
2. True
3. false

b. Explain why "10" + 5 does not produce the number 15. ***(2 marks)***

When you use + and one side is a string, JavaScript joins them as text instead of adding them as numbers. It turns the 5 into "5" and sticks it onto "10", giving the string "105" instead of the number 15.

c. Explain the difference between == and ===. ***(2 marks)***

- == is loose equality which converts both sides to the same type before comparing, so 5 == "5" is true.
- === is strict equality which compares both the value and the type, with no conversion, so 5 === "5" is false.

d. Rewrite the first statement so that it produces the number 15. ***(1 mark)***

```
const total = Number(“7”) + 8
```

10. Study the following functions: ***(8 marks total)***

```
function calculateTotal(price, quantity = 1) {
  return price * quantity;
}
  const applyDiscount = (total, percentage) => {
  return total - total * (percentage / 100);
};
  const orderTotal = calculateTotal(150, 3);
const finalTotal = applyDiscount(orderTotal, 10);
```

a. Identify the parameters of calculateTotal(). ***(2 marks)***

The parameters are price and quantity

b. Explain the purpose of the default value assigned to quantity. ***(1 mark)***

If calculateTotal() is called without a value for quantity, it automatically uses 1 instead, so the function still works properly rather than returning NaN or undefined.

c. Explain what the return keyword does. ***(1 mark)***

return sends a value back out of the function to wherever it was called from, and immediately stops the function running.

d. State the value stored in orderTotal. ***(1 mark)***

orderTotal = calculateTotal(150, 3) = 150 × 3 = 450

e. State the value stored in finalTotal. ***(1 mark)***

finalTotal = applyDiscount(450, 10) = 450 − (450 × 10 / 100) = 450 − 45 = 405.

f. Explain one difference between a function declaration and an arrow function. ***(2 marks)***

One difference is that a function declaration is hoisted, meaning it can be called before the line it's defined on. An arrow function assigned to a const or let is not hoisted the same way, so it must be defined before it's called.

11. Study the following array: ***(10 marks total)***

```
const tasks = [
  { title: "Create wireframes", completed: true, hours: 3 },
  { title: "Develop login form", completed: false, hours: 5 },
  { title: "Test application", completed: true, hours: 2 },
  { title: "Write documentation", completed: false, hours: 4 }
];
```

Write JavaScript code that:

a. Uses a loop to display the title of every task. ***(2 marks)***

```
for (const task of tasks) {
console.log(task.title);
}
```

b. Uses a conditional statement to display only completed tasks. ***(2 marks)***

```
for (const task of tasks) {
 if (task.completed)
  {console.log(task.title);
 }
}
```

c. Counts the number of completed tasks. ***(2 marks)***

```
const completedCount = tasks.filter(task => task.completed).length;
```

d. Calculates the total number of hours for all tasks. ***(2 marks)***

```
const totalHours = tasks.reduce((sum, task) => sum + task.hours, 0)
```

e. Displays an appropriate message if no tasks are available. ***(2 marks)***

```
if (tasks.length === 0) {
  console.log("No tasks are available.");
} else {
  // display or process the tasks as normal
}
```

15. Study the following statement: ***(4 marks total)***

document.cookie = "theme=dark; max-age=3600; path=/";

a. Explain what information the cookie stores. ***(1 mark)***
it stores a cookie called theme with the value dark. a small piece of data is saved in the users browser, in this case remembering that the user chose the dark theme, so it can be recalled next time

b. Explain the purpose of max-age=3600. ***(1 mark)***
It sets how long the cookie lasts, in seconds. The cookie automatically expires and is deleted 3600 seconds (1 hour) after it was created.

c. Explain the purpose of path=/. ***(1 mark)***
It says which part of the site the cookie can be used on. path=/ means the cookie is available on every page of the site, not just one folder or page.

d. Write a JavaScript statement that displays the available cookies. ***(1 mark)***
console.log(document.cookie);

Testing and Problem-Solving

16. A registration form contains the following fields: name; email address; password; and age. ***(15 marks)***

Develop five test cases that could be used to test the form. Each test case must include the input or condition being tested and the expected result. Your test cases must include at least one valid submission, one missing value, one invalid email address, and one boundary-value test.

Test Case: Input / Condition Tested	Expected Result
Name: "Chad Teixeira", Email: "Chad@example.com", Password: "Secure@123", Age: 20 (all fields filled in correctly)	The form is accepted: no error messages are shown, and the data is saved successfully.
Name field left blank; Email, Password and Age all filled in correctly	The form does not submit. An error such as "Name is required" is shown, and the user is asked to complete the field.
Email entered as "thabo.example.com" (missing the @ symbol); other fields valid	The form does not submit. An error such as "Please enter a valid email address" is shown.
Age entered as 17, one year below the minimum allowed age of 18; other fields valid	The form does not submit. An error such as "You must be at least 18 years old to register" is shown. (At the boundary itself, Age = 18, the form should be accepted.)
Password entered as "123" (under the required minimum of 8 characters); other fields valid	The form does not submit. An error such as "Password must be at least 8 characters long" is shown.

Academic Integrity and Authenticity Declaration

I declare that:

- the evidence submitted for assessment accurately represents my own contribution;
- I have acknowledged tutorials, libraries, code examples, artificial-intelligence assistance and other external sources;
- I understand the JavaScript, Firebase and GitHub work that I am presenting;
- I can explain, test and modify the code attributed to me;
- I have not stored passwords, private credentials or confidential data in the repository;
- I understand that if I cannot demonstrate understanding OR provide enough evidence, my project could not be accepted and

therefore I will not pass the FISA (Final Integrated Supervised Assessment).

Learner names: Chad Teixeira, Brian Mncube, Modise Matjila

Learner signature: C.T B.M M.M

Date: 31/08/2026

Assessor name:

Assessor signature:

Date